Tasks on docker-02

1. Create a Tomcat Container on Port 8080 and Deploy a Sample Application

Implementation Steps

1. Pulled the Tomcat Docker image from Docker Hub.

   sudo docker pull tomcat:latest

2. Login to tomcat and download sample war file in webapps.

   sudo docker exec -it <cont.ID> /bin/bash

   cd /usr/local/tomcat/webapps

   wget <sample_war_url>

3. Created and started the Tomcat container, mapping host port 8080 to container port 8080.

   sudo docker run -d \

   --name tomcat-8080 \

   -p 8080:8080 \

   tomcat:latest

4. Verified the container status and application response.

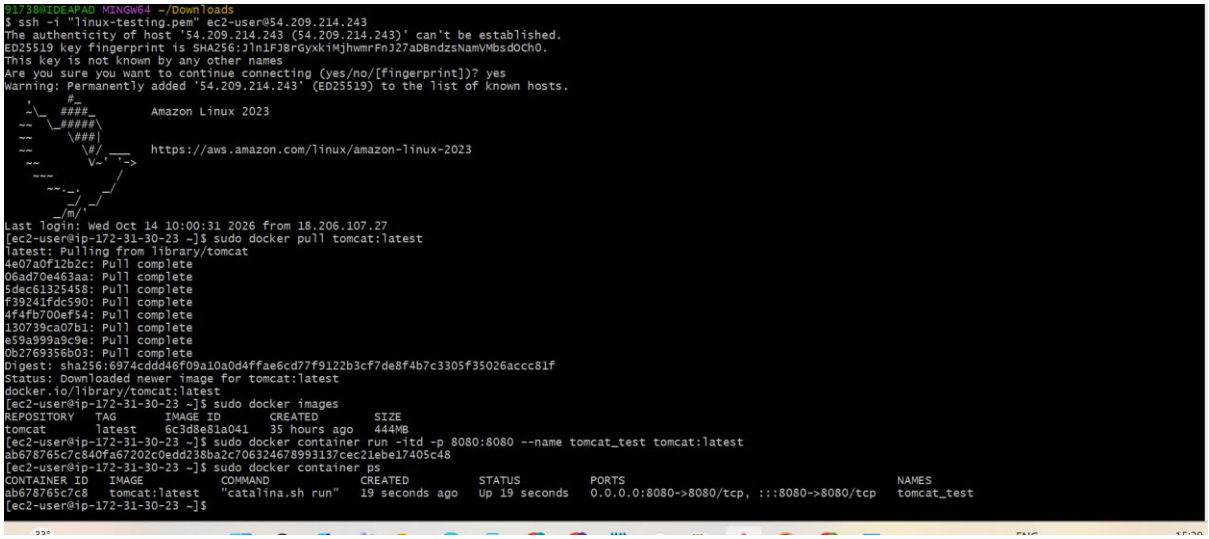
5. sudo docker ps

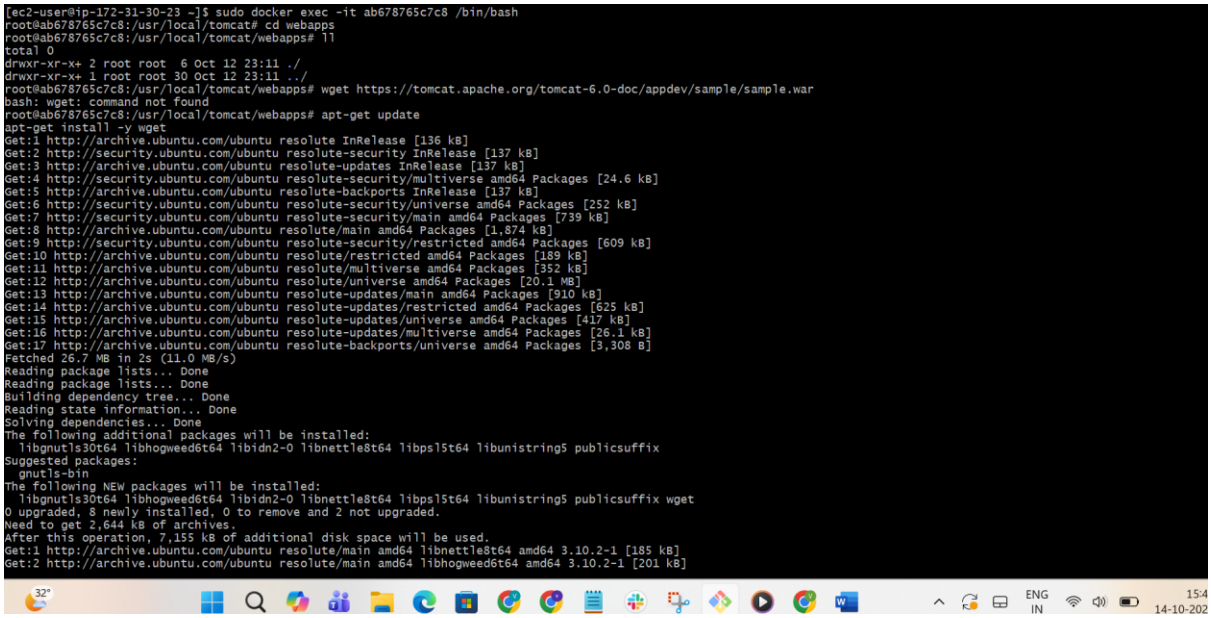
curl http://localhost:8080/sample/

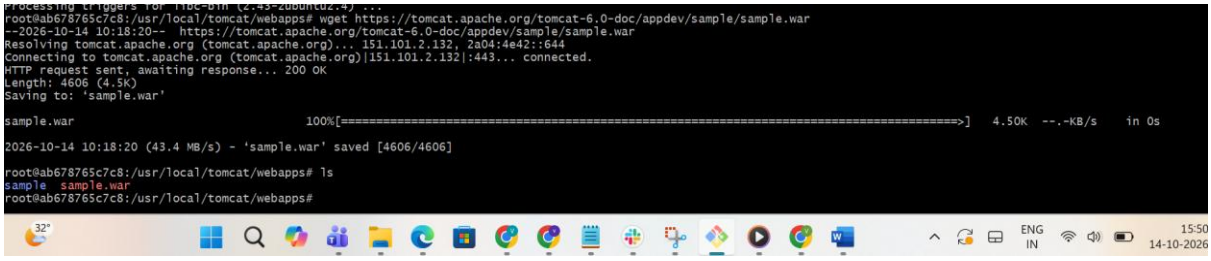
6. Accessed the application through http://<EC2-Public-IP>:8080/sample/ after allowing inbound TCP port 8080 in the EC2 security group.

Result

Successfully created a Tomcat container on port 8080 and deployed a sample war application.









---

2. Create a Docker Volume and Deploy a Tomcat Container on Port 8081

Implementation Steps

1. Created a Docker volume named tomcat-volume.

   sudo docker volume create tomcat-volume

2. Verified the volume.

   sudo docker volume ls

3. Created a Tomcat container, mapped host port 8081 to container port 8080, and mounted the Docker volume at the web applications directory.

sudo docker run -d \

--name tomcat-volume-container \

-p 8081:8080 \

-v tomcat-volume:/usr/local/tomcat/webapps \

tomcat:latest

4. Copied the sample WAR file from the EC2 host into the container. The WAR file was obtained during the application deployment exercise.
5. sudo docker cp ~/sample.war \

   tomcat-volume-container:/usr/local/tomcat/webapps/sample.war
6. Checked the container status and volume configuration.

   sudo docker ps

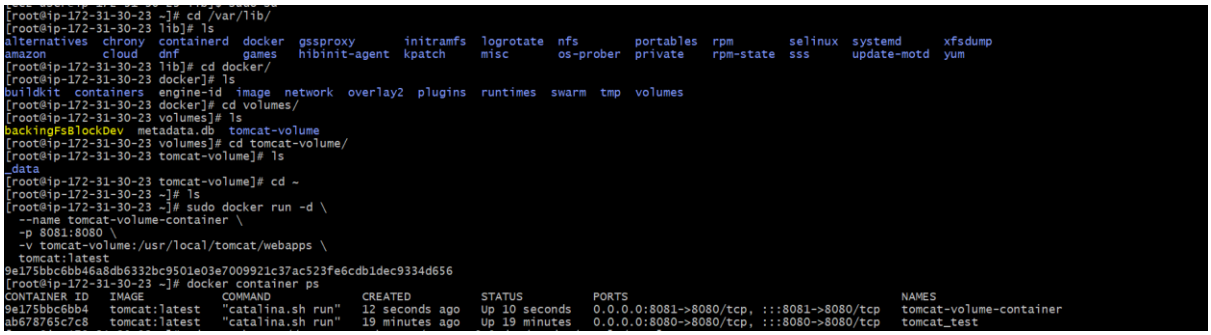
   sudo docker volume inspect tomcat-volume
7. Checked the Tomcat logs to verify application deployment.

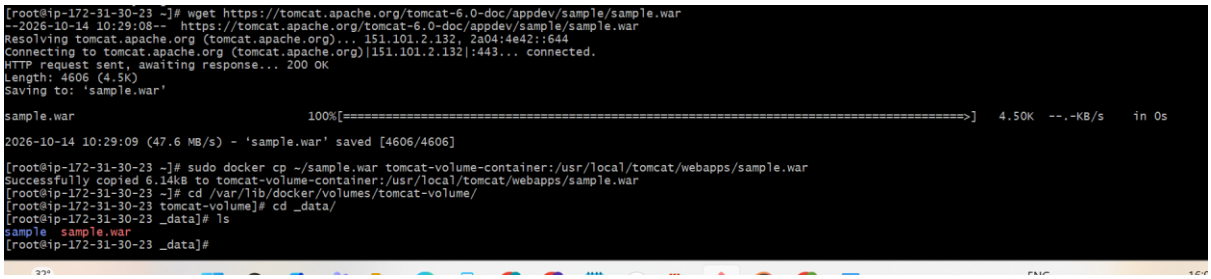
   sudo docker logs tomcat-volume-container
8. Tested the application.

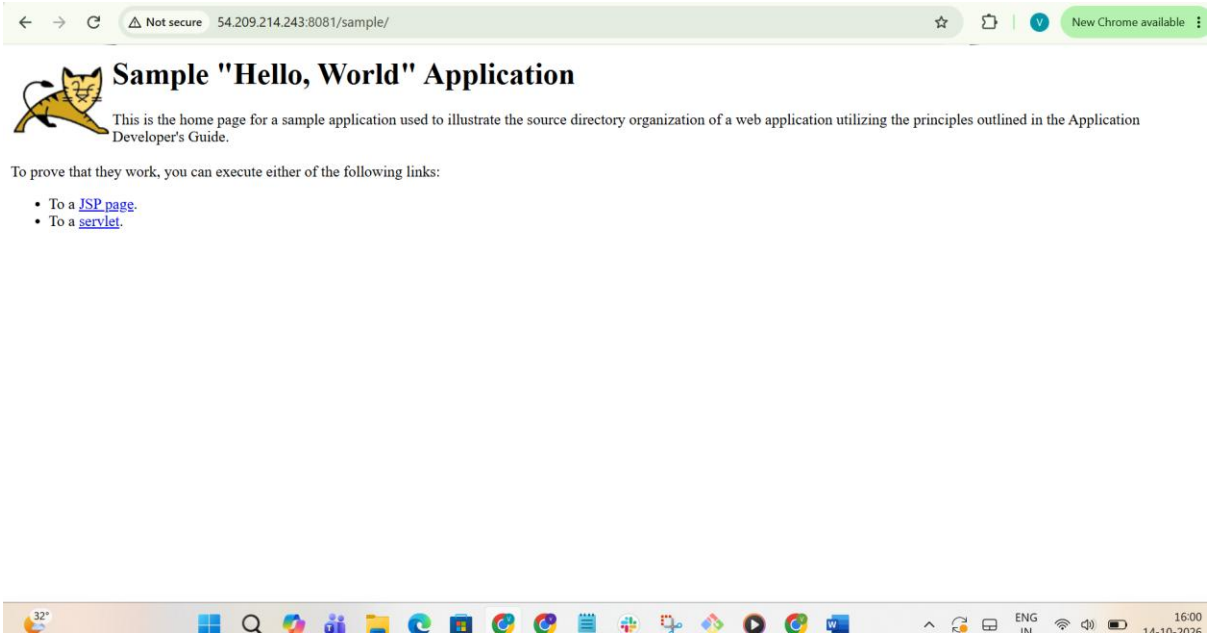
   curl -I http://localhost:8081/sample/

Result

Successfully created a Docker volume, mounted it to a Tomcat container, deployed a sample application.







---

3. Limit the Nginx Container Memory to 500 MB

Implementation Steps

1. Pulled the Nginx Docker image.

   sudo docker pull nginx:stable

2. Created and started an Nginx container with a memory limit of 500 MiB and mapped host port 8082 to container port 80.

   sudo docker run -d \

   --name nginx-limited \

   -p 8082:80 \

   --memory=500m \

   nginx:stable

3. Verified that the container was running.

   sudo docker ps

4. Inspected the configured memory limit.

   sudo docker inspect nginx-limited \

   --format='Memory limit: {{.HostConfig.Memory}} bytes'

5. Verified container resource usage.

   sudo docker stats nginx-limited

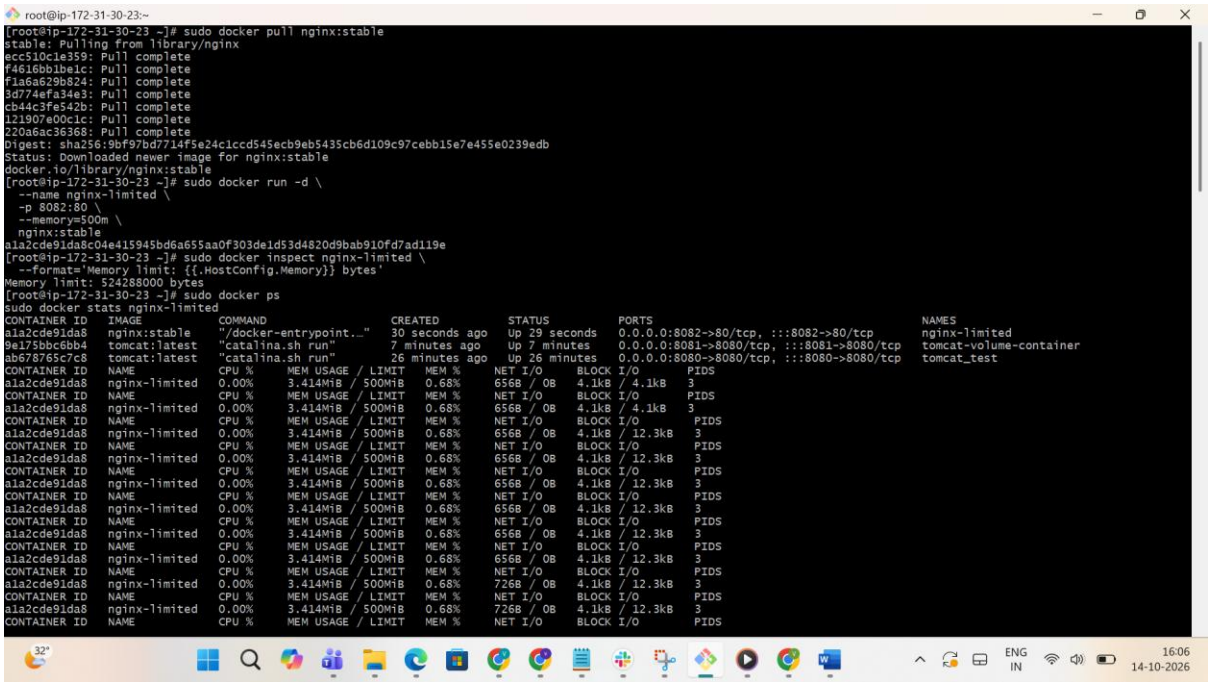
6. Tested the Nginx HTTP response.

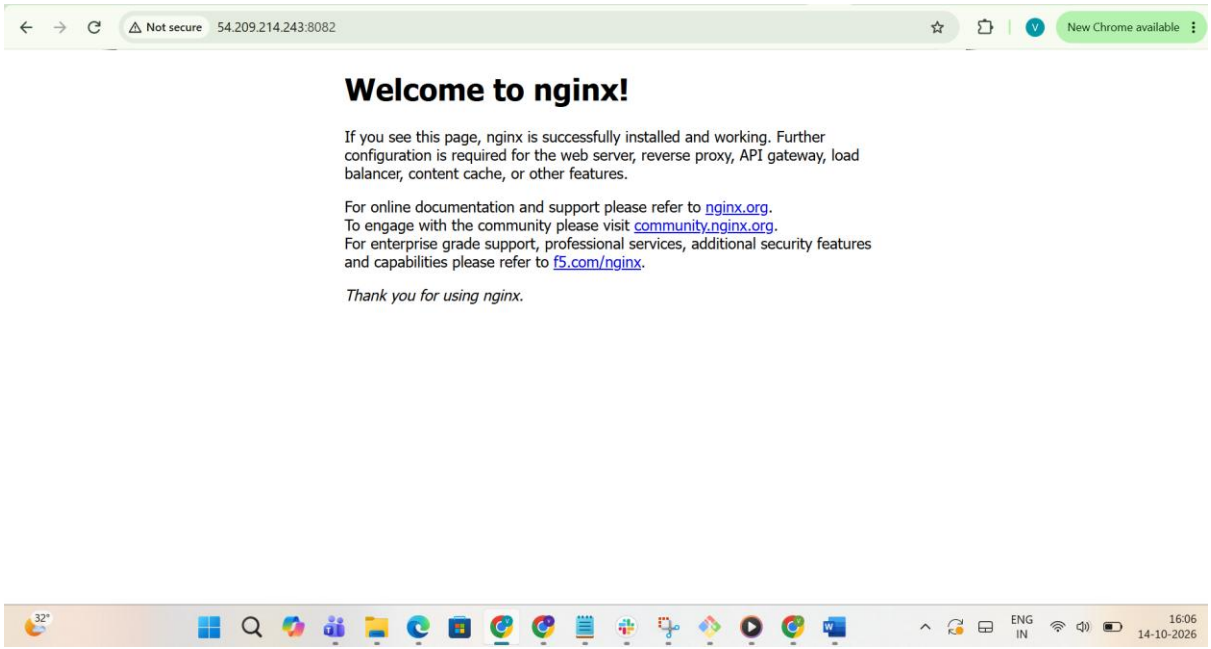
curl -I http://localhost:8082

7. Accessed the Nginx default page at http://<EC2-Public-IP>:8082 after allowing inbound TCP port 8082 in the EC2 security group.

Result

Successfully deployed an Nginx container with a 500 MiB memory limit and verified its resource configuration and HTTP response.





---

4. Create a Custom Docker Image Using Amazon Linux and Nginx

Implementation Steps

1. Created a project directory.

   mkdir -p ~/custom-nginx

   cd ~/custom-nginx

2. Created an index.html file containing the custom web page.

   <!DOCTYPE html>

   <html>

   <head>

   <title>Custom Nginx Docker</title>

   </head>

   <body>

   <h1>Welcome to Amazon Linux Nginx!</h1>

   My custom Docker image is running successfully.

   </body>

   </html>

3. Created a file named Dockerfile with the following instructions.

   FROM amazonlinux:latest

   MAINTAINER="Vamshi"

   RUN dnf install -y nginx && dnf clean all

   COPY index.html /usr/share/nginx/html/index.html

   EXPOSE 80

   CMD ["nginx", "-g", "daemon off;"]

4. Used amazonlinux:latest as the base image.
5. Added maintainer information using the LABEL instruction.
6. Installed Nginx using the dnf package manager.
7. Copied index.html into the Nginx document root.
8. Declared port 80 using the EXPOSE instruction.
9. Configured Nginx to run in the foreground using the CMD instruction.
10. Built the custom Docker image.

    sudo docker build -t custom-nginx:latest .

11. Verified that the image was created.

sudo docker images

12. Created and started a container from the custom image, mapping host port 80 to container port 80.

    sudo docker run -d \

     --name custom-nginx \

     -p 80:80 \

    custom-nginx:latest

13. Verified the container and tested the custom HTML page.

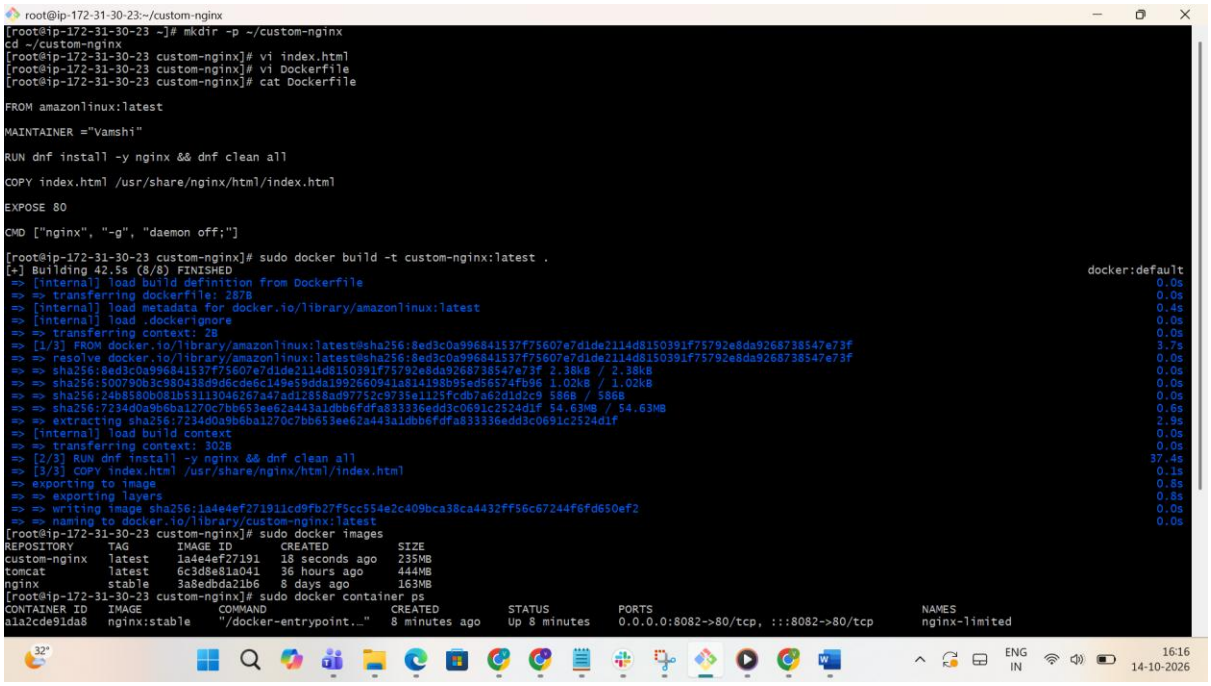
    sudo docker ps

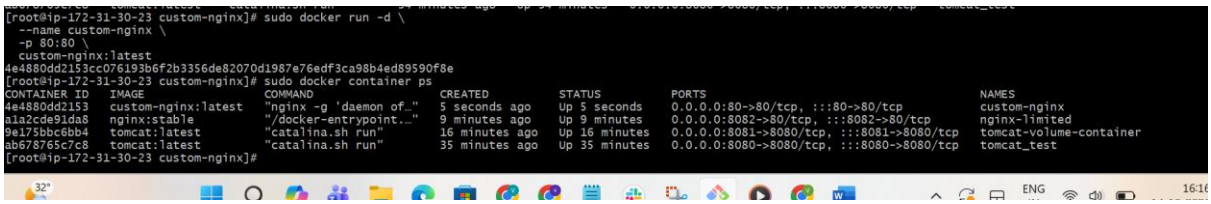
    curl http://localhost:80

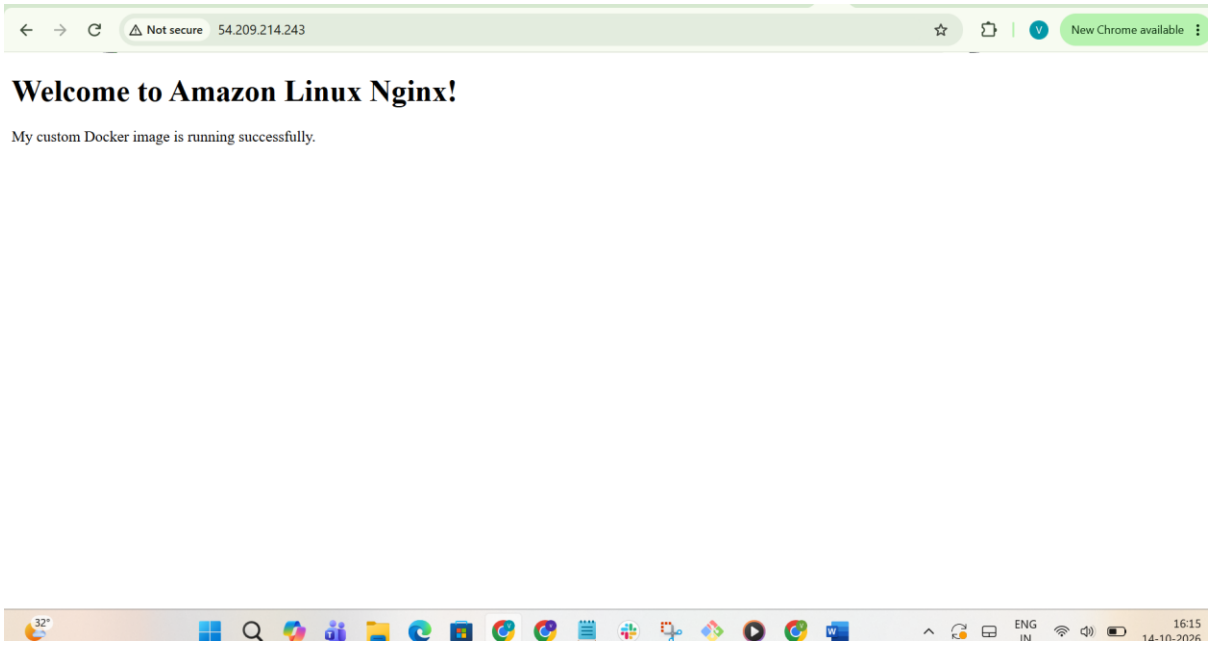
14. Accessed the application at http://<EC2-Public-IP>:80 after allowing inbound TCP port 80.

Result

Successfully created and tested a custom Docker image using Amazon Linux, Nginx, a custom HTML page, and the required Dockerfile instructions.







---

5. Push the Custom Docker Image to Docker Hub

Implementation Steps

1. Created a Docker Hub account or signed in to an existing account.
2. Created a repository named custom-nginx in Docker Hub and selected the required visibility.
3. Authenticated Docker to Docker Hub using the account username.

sudo docker login -u YOUR_DOCKERHUB_USERNAME

4. Entered the Docker Hub password or personal access token when prompted.
5. Tagged the existing custom image with the Docker Hub username and repository name.
6. sudo docker tag custom-nginx:latest \

YOUR_DOCKERHUB_USERNAME/custom-nginx:latest

7. Verified the image tag.

sudo docker images

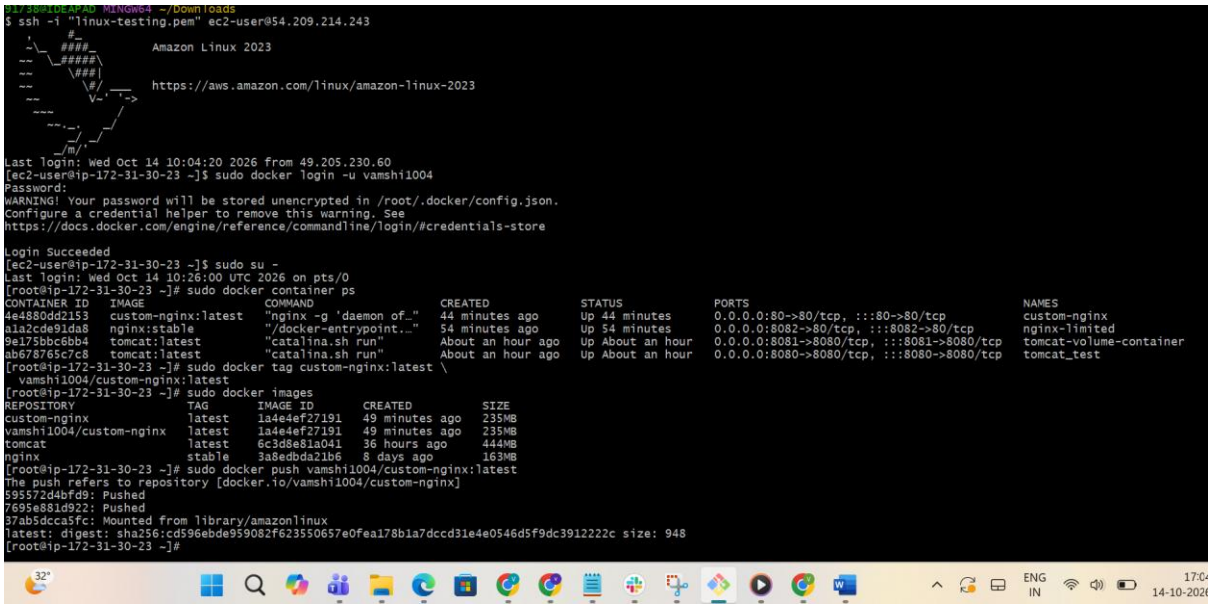
8. Pushed the tagged image to Docker Hub.
9. sudo docker push \

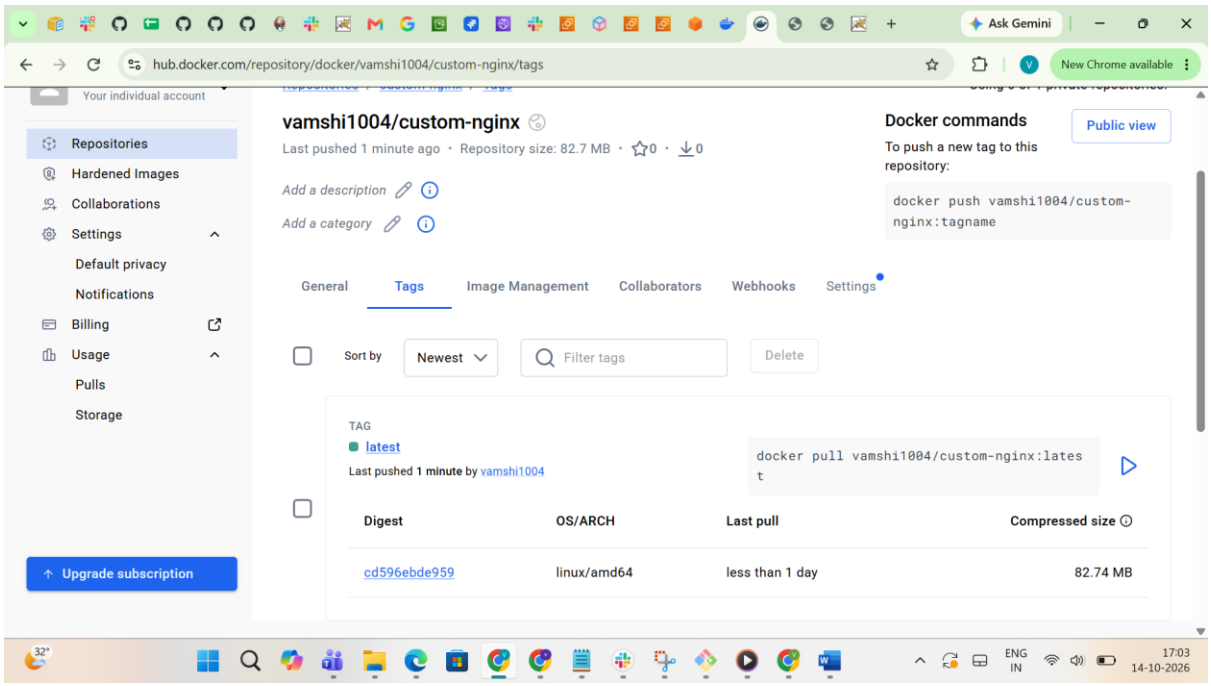
YOUR_DOCKERHUB_USERNAME/custom-nginx:latest

10. Opened Docker Hub and verified that the latest tag was available in the custom-nginx repository.

Result

Successfully authenticated to Docker Hub, tagged the custom Nginx image, and pushed it to the remote Docker Hub repository.





---

6. Push the Custom Docker Image to AWS Elastic Container Registry (ECR)

Implementation Steps

1. Verified the AWS CLI installation.

   aws --version

2. Configured an IAM role named EC2-ECR-Push-Role for the EC2 instance and attached the AmazonEC2ContainerRegistryPowerUser policy.

3. Verified the active AWS identity.

   aws sts get-caller-identity

4. Created a private ECR repository named custom-nginx in the Mumbai Region (ap-south-1) using the AWS Console.

5. Set the AWS Region and retrieved the account ID.

   REGION=ap-south-1

   ACCOUNT_ID=$(aws sts get-caller-identity \

   --query Account --output text)

6. Authenticated Docker to the ECR registry using the AWS CLI.

   aws ecr get-login-password --region ap-south-1 | \

   docker login \

    --username AWS \

    --password-stdin \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com

7. Tagged the existing custom Nginx image with the ECR registry URI.

   docker tag custom-nginx:latest \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest

8. Pushed the image to the ECR repository.

   docker push \

   281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest

9. Verified the uploaded image using the AWS CLI.

   aws ecr describe-images \

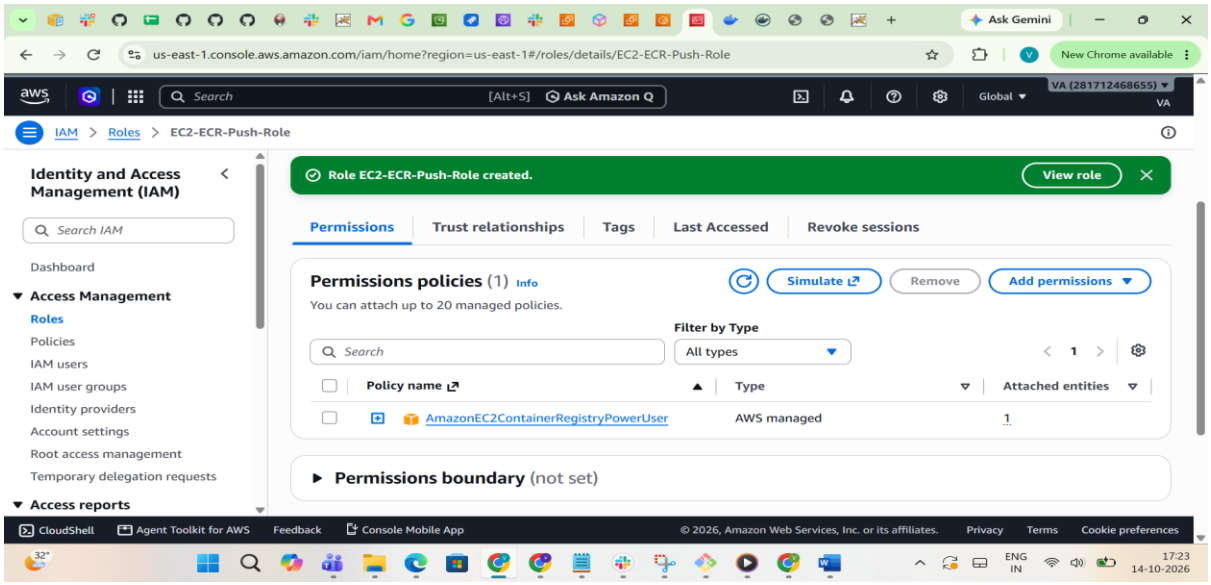
    --repository-name custom-nginx \

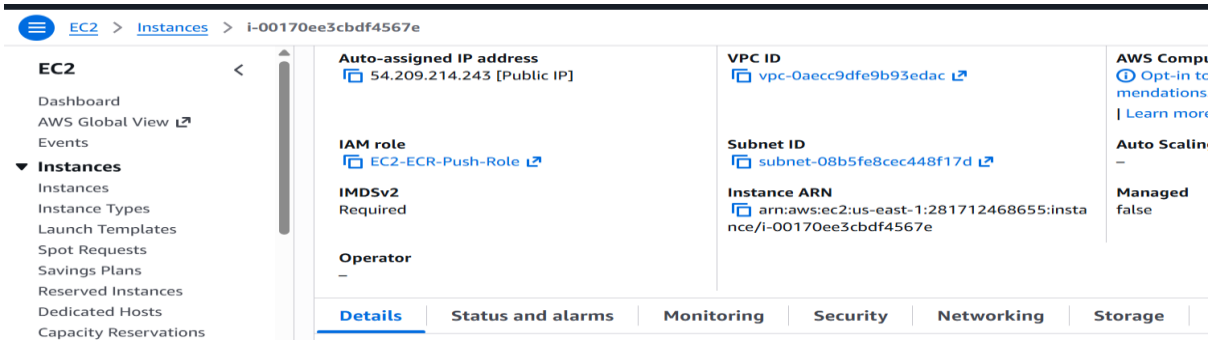
   --region ap-south-1

10. Opened the AWS Console and navigated to **Amazon ECR → Private registry → Repositories → custom-nginx → Images** to verify the uploaded latest image tag.

Result

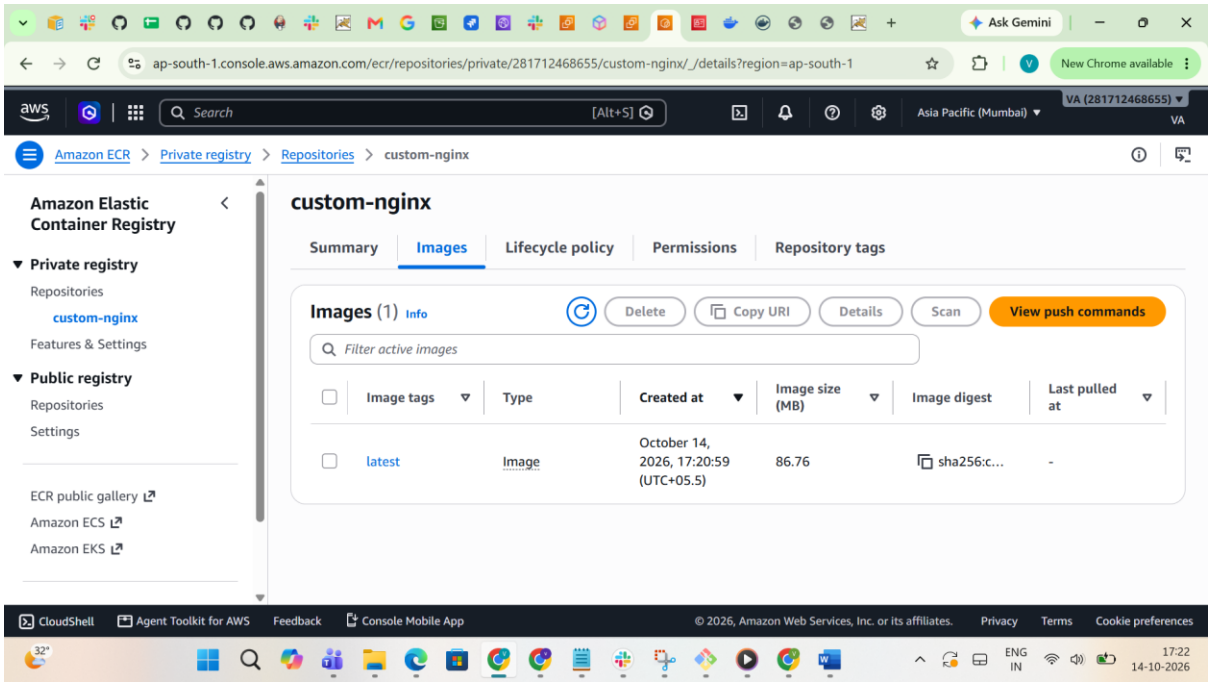
Successfully authenticated Docker to AWS ECR, tagged the custom Nginx image, pushed it to the private ECR repository, and verified the image in the Mumbai Region.






```
[root@ip-172-31-30-23 ~]# aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
281712468655.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[root@ip-172-31-30-23 ~]# docker images
REPOSITORY                                                TAG       IMAGE ID       CREATED             SIZE
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx   latest    1a4e4ef27191   About an hour ago   235MB
custom-nginx                                              latest    1a4e4ef27191   About an hour ago   235MB
vamshi1004/custom-nginx                                   latest    1a4e4ef27191   About an hour ago   235MB
tomcat                                                    latest    6c3d8e81a041   37 hours ago        444MB
nginx                                                     stable    3a8edbda21b6   8 days ago          163MB
[root@ip-172-31-30-23 ~]# docker tag custom-nginx:latest \
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
[root@ip-172-31-30-23 ~]# docker push \
281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx:latest
The push refers to repository [281712468655.dkr.ecr.ap-south-1.amazonaws.com/custom-nginx]
595572d4bfd9: Pushed
7695e881d922: Pushed
87ab5dcca5fc: Pushed
latest: digest: sha256:cd596ebde959082f623550657e0fea178b1a7dccd31e4e0546d5f9dc3912222c size: 948
[root@ip-172-31-30-23 ~]# aws ecr describe-images \
  --repository-name custom-nginx \
  --region ap-south-1
{
    "imageDetails": [
        {
            "registryId": "281712468655",
            "repositoryName": "custom-nginx",
            "imageDigest": "sha256:cd596ebde959082f623550657e0fea178b1a7dccd31e4e0546d5f9dc3912222c",
            "imageTags": [
                "latest"
            ],
            "imageSizeInBytes": 86762510,
            "imagePushedAt": "2026-10-14T11:50:59.226000+00:00",
            "imageManifestMediaType": "application/vnd.docker.distribution.manifest.v2+json",
            "artifactMediaType": "application/vnd.docker.container.image.v1+json",
            "imageStatus": "ACTIVE"
        }
    ]
}
[root@ip-172-31-30-23 ~]#
```


Ask Gemini
ap-south-1.console.aws.amazon.com/ecr/repositories/private/281712468655/custom-nginx/_/details?region=ap-south-1
New Chrome available
Search
[Alt+S]
Asia Pacific (Mumbai)
VA (281712468655)
VA
Amazon ECR
Private registry
Repositories
custom-nginx
Amazon Elastic Container Registry
Private registry
Repositories
custom-nginx
Features & Settings
Public registry
Repositories
Settings
ECR public gallery
Amazon ECS
Amazon EKS
custom-nginx
Summary
Images
Lifecycle policy
Permissions
Repository tags
Images (1) Info
Delete
Copy URI
Details
Scan
View push commands
Filter active images
Image tags
Type
Created at
Image size (MB)
Image digest
Last pulled at
latest
Image
October 14, 2026, 17:20:59 (UTC+05.5)
86.76
sha256:c...
-
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
ENG
IN
17:22
14-10-2026